

Partition-Based Clustering

Numerical Worksheet

INT396 • Unsupervised Learning • Unit II

School of Computer Science & Engineering

Solutions copy

Carried in from Unit I you already know how to compute a distance between two points. This worksheet *uses* that skill and does not retest it: wherever a distance appears, apply $d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$ and move on. Every formula in the next section is new, and all of it belongs to Unit II.

Contents

1	The Unit II formulas	1
2	Part A — Clustering fundamentals and assumptions	4
3	Part B — The k-Means objective	9
4	Part C — Initialisation and convergence	15
5	Part D — k-Medoids and MiniBatch k-Means	20
6	Part E — Standardisation and scaling	24
7	Part F — Cluster validation	27

1 The Unit II formulas

A partition the object every algorithm in this unit produces: k groups that between them account for every point exactly once.

$$C_i \cap C_j = \emptyset \quad (i \neq j), \quad C_1 \cup C_2 \cup \dots \cup C_k = X$$

No point belongs to two clusters, and no point is left out.

Soft, or fuzzy, membership the same partition written as weights rather than labels. Each point carries one weight per cluster and they form a distribution.

$$u_{ij} = \frac{\frac{1}{d_{ij}^{2/(m-1)}}}{\sum_{c=1}^k \frac{1}{d_{ic}^{2/(m-1)}}} \quad \text{and} \quad \sum_{j=1}^k u_{ij} = 1$$

where

d_{ij} the distance from point i to the centre of cluster j

m the fuzziness, $m > 1$; the standard choice is $m = 2$

Read it as *one over the distance, share of the total*: near centres give big reciprocals and so big weights. With the usual $m = 2$ the exponent $2/(m - 1)$ is just 2, and the whole thing becomes the much friendlier

$$u_{ij} = \frac{\frac{1}{d_{ij}^2}}{\frac{1}{d_{i1}^2} + \frac{1}{d_{i2}^2} + \cdots + \frac{1}{d_{ik}^2}}$$

The hard label is the cluster with the largest u_{ij} , and everything else is discarded.

Inertia, or within-cluster sum of squares the quantity k-Means exists to minimise.

$$\text{WCSS} = \sum_{j=1}^k \sum_{x \in C_j} [d(x, \mu_j)]^2$$

where

C_j the set of points assigned to cluster j

μ_j the centroid of cluster j , that is the mean of its members

$d(x, \mu_j)$ the ordinary distance from the point to that centroid

In words: go through the clusters one at a time; inside each, measure every point's distance to its own centre, square it, and keep a running total.

Centroid update the centre of a cluster is the average of its members — add the members up, divide by how many there are.

$$\mu_j = \frac{x_1 + x_2 + \cdots + x_{n_j}}{n_j} \quad \text{where } n_j = \text{number of points in } C_j$$

k-Means++ selection probability each remaining point gets a share of the draw proportional to its squared distance from the nearest centre already chosen.

$$\text{Pr}(x) = \frac{[D(x)]^2}{[D(x_1)]^2 + [D(x_2)]^2 + \cdots + [D(x_n)]^2}$$

where

$D(x)$ the distance from x to the nearest centre already chosen

k-Medoids cost the medoid is the cluster member with the smallest total distance to all the others. Test each member in turn and keep the winner.

$$\text{cost}(m) = d(x_1, m) + d(x_2, m) + \cdots + d(x_n, m), \quad m \text{ must be one of the } x_i$$

Two differences from k-Means: the distances are *not* squared, and the centre is not free to be any position — it has to be one of your actual data points.

Standardisation, and min-max scaling

$$z = \frac{x - \mu}{\sigma}, \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Silhouette coefficient scored one point at a time, and the only measure here that can tell you a point is in the *wrong* cluster.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

where

$a(i)$ the mean distance from i to the other members of its own cluster

$b(i)$ the mean distance from i to the members of the nearest *other* cluster

It runs from -1 to $+1$. The silhouette of a whole clustering is the mean of $s(i)$ over every point.

Davies–Bouldin Index

$$R_{ij} = \frac{S_i + S_j}{M_{ij}} \quad \text{then} \quad \text{DB} = \frac{R_1^{\max} + R_2^{\max} + \dots + R_k^{\max}}{k}$$

where

S_j the mean distance from the members of cluster j to μ_j

M_{ij} the distance between centroids μ_i and μ_j

$R_j^{\max}$ the largest R_{ij} that cluster j makes with any other cluster — its worst partner

Do it in two steps and it is easy: first work out the ratio R_{ij} for every pair, then give each cluster its *worst* ratio and average those k numbers. Lower is better. Unlike inertia it has a genuine minimum over k , so it can be optimised rather than merely inspected.

The worksheet runs in six parts, following the Unit II syllabus in order. Part A is clustering fundamentals — what a partition is, hard against soft, and the assumptions k -Means makes; Part B is the k -Means objective; Part C is initialisation and convergence; Part D is k -Medoids and MiniBatch; Part E is standardisation and scaling; Part F is cluster validation.

How every part is laid out. All six parts follow the same four steps, in the same order, every time:

1	The concept	what the idea is, in words, before any symbol is used in anger.
2	The picture	the same idea drawn, so you can see what the formula is doing.
3	How it differs	the one method it is most often confused with, set beside it.
4	The numerical	one fully worked calculation on numbers small enough to check by hand.

Then, and only then, the **questions**. Read all four steps before attempting anything. The worked numerical in step 4 deliberately uses different numbers from the questions, so it teaches the method without giving any answer away.

2 Part A — Clustering fundamentals and assumptions

1. The concept

What a partition is every method in this unit returns the same kind of object: a **partition**. Each point is handed exactly one label, subject to two rules — the clusters must not overlap, and between them they must account for every point. Break the first and a customer is in two segments at once; break the second and a customer is in none.

Hard and soft assignment a **hard** clustering issues one label per point: customer 137 is in cluster 2, full stop. A **soft** or *fuzzy* clustering issues a weight per cluster instead — 0.55 cluster 2, 0.30 cluster 4, 0.15 elsewhere — and the weights must sum to 1. The hard label is simply the largest weight with everything else thrown away, which is exactly why the hard label cannot tell a borderline customer from a certain one.

What k-Means assumes k-Means does not look for groups in general. It looks for one specific kind of group, because of how its objective is written, and it assumes without asking that your clusters are

- (i) **roughly spherical** — equally wide in every direction, since each cluster is described by a single centre point;
- (ii) **similar in spread** — a tight cluster beside a loose one will steal the loose one's outer members;
- (iii) **similar in size** — a cluster of 20 beside one of 300 tends to be swallowed, because moving the boundary costs the large one little;
- (iv) **separable by straight lines** — the boundary between two centres is always their perpendicular bisector, so every region k-Means can carve out is straight-edged and convex.

These matter because your data does not have to obey them, and when it does not *k-Means still returns an answer*. It never refuses and it never warns.

2. The picture

Two things are worth seeing rather than reading. First, what the four assumptions look like when they fail — Figure 1. Second, what a soft membership keeps that a hard label throws away — Figure 2.

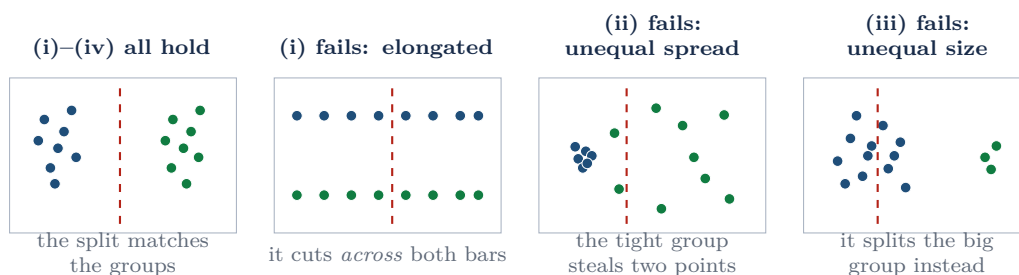


Figure 1: The same algorithm, $k = 2$, on four datasets. *Colour* shows the real groups; the *dashed line* is the boundary k-Means produces. Only in the first panel do the two agree — and in the other three nothing in the output says so.



Figure 2: P sits almost equally far from both centres. The soft record keeps that fact; the hard label destroys it, and P becomes indistinguishable from a point deep inside A.

3. How it differs — a partition against the alternatives

Being a *partition* method is a real restriction, and it is what separates this unit from Unit III. Hierarchical clustering returns a whole tree of nested groupings rather than one partition, and DBSCAN is allowed to label a point as **noise** and place it in no cluster at all.

	k-Means (Unit II)	Hierarchical (Unit III)	DBSCAN (Unit III)
What it returns	one partition	a tree of nested partitions	clusters plus a noise set
Can a point end up in no cluster?	no — every point is labelled	no	yes , that is the point
Must you fix k first?	yes	no — you cut the tree afterwards	no — it follows from density
Shapes it can find	convex, roughly round	depends on the linkage	any shape at all

The row that matters most is the second. k-Means has no way of saying “this point belongs nowhere” — so an outlier is not merely tolerated, it is *assigned*, and it drags a centroid with it. Every weakness in Parts B to D traces back to that one sentence.

4. The numerical

Worked example 2.1 — turning distances into soft memberships

A point lies at distances 1, 2 and 4 from three cluster centres. Take the standard fuzziness $m = 2$, so the exponent $2/(m - 1)$ is simply 2 and each weight is *one over the distance squared*.
Step 1 — square each distance, then turn it upside down

$$\frac{1}{1^2} = \frac{1}{1} = 1, \quad \frac{1}{2^2} = \frac{1}{4} = 0.25, \quad \frac{1}{4^2} = \frac{1}{16} = 0.0625$$

Step 2 — total them

$$\Sigma = 1 + 0.25 + 0.0625 = 1.3125$$

Step 3 — divide each by the total

$$u_1 = \frac{1}{1.3125} = \mathbf{0.7619}, \quad u_2 = \frac{0.25}{1.3125} = \mathbf{0.1905}, \quad u_3 = \frac{0.0625}{1.3125} = \mathbf{0.0476}$$

Check, then read the answer

The three sum to 1.0000, as memberships must. The hard label is cluster 1, and the 0.7619 beside it is the piece of information the hard label throws away: this point is comfortably placed, and had the weights come out 0.40/0.38/0.22 the same hard label would have been a coin toss.

Questions

Solutions follow each question, with full working.

A1 Is it a partition? A dataset holds six points $X = \{P_1, P_2, P_3, P_4, P_5, P_6\}$. Three candidate groupings are proposed:

$$(a) \{P_1, P_2, P_3\}, \{P_4, P_5, P_6\}$$

$$(b) \{P_1, P_2, P_3\}, \{P_3, P_4, P_5, P_6\}$$

$$(c) \{P_1, P_2\}, \{P_4, P_5\}$$

State which are valid partitions. For each failure name the rule broken. Then say which of the three a density-based method such as DBSCAN could legitimately return, and why that is not a contradiction.

Solution.

(a)

Valid. The two sets share no point and together they contain all six.

(b)

Not a partition. P_3 appears in both sets, so $C_1 \cap C_2 = \{P_3\} \neq \emptyset$ — the clusters are not disjoint. A customer cannot be in two segments at once; asked which campaign to send them, the answer would be undefined.

(c)

Not a partition. P_3 and P_6 appear in neither set, so $C_1 \cup C_2 \neq X$ — the clusters are not exhaustive.

The DBSCAN question

Grouping (c). DBSCAN is not a partition method in this strict sense: it is allowed to declare a point *noise* and assign it to no cluster, which is exactly what (c) does with P_3 and P_6 . That is a deliberate design difference, not a fault — and it is precisely why DBSCAN can find outliers while k-Means cannot. k-Means has no such option: every point gets a label whether it deserves one or not.

A2 Which assumption fails? Figure 1 shows four datasets, each clustered with $k = 2$. For each of the last three panels, name the assumption from the list (i)–(iv) that is violated, and say in one sentence what k-Means returns instead of the grouping a human would draw. Then answer: which assumption does a dataset shaped like two concentric rings violate?

Solution.

Panel 2 — elongated

Assumption (i), roughly spherical, fails. Each group is far longer than it is tall, so the two points at opposite ends of the same bar are further apart than points on different bars. k-Means cuts vertically *across* both bars, returning left-half and right-half instead of top and bottom.

Panel 3 — unequal spread

Assumption (ii) fails. The tight group's centroid barely moves, so the boundary sits close to it and the loose group's nearest members are closer to the tight centroid than to their own. The wide cluster loses its inner edge.

Panel 4 — unequal size

Assumption (iii) fails. Isolating three points as their own cluster lowers inertia far less than splitting the crowd of twelve does, so the objective prefers to cut the large group in two and let the three strays join one half.

Concentric rings

Assumption (iv), separability by straight lines. Every region k-Means can produce is bounded by perpendicular bisectors and is therefore convex. A ring is not convex, and its own centre does not even lie inside it, so no choice of centroids describes it. Note that this is a property of the *objective function*, not of any implementation — no amount of restarting or tuning repairs it, and Unit III exists partly for this reason.

A3 Hard and soft assignment (a) Under soft clustering a point has membership 0.15 in cluster A and 0.55 in cluster B, with only one further cluster C available. Find its membership in C, give the hard assignment, and say what is lost by making it.

(b) A second point lies at distances 2, 3 and 6 from the three cluster centres. Using $m = 2$, compute its three memberships and its hard label.

(c) Which of the two points would you rather act on, and why? Figure 2 is the picture for part (a).

Solution.

(a)

Memberships must sum to 1, so $w_C = 1 - 0.15 - 0.55 = \mathbf{0.30}$. The hard assignment is **B**, since 0.55 is the largest.

What is lost is the knowledge that this is a borderline point: 0.55 against 0.30 is close to a tie. Once the hard label is issued, the point is indistinguishable from one with membership 0.99 in B — and it is exactly these boundary customers whose behaviour a campaign is most likely to misjudge.

(b) *One over each distance, squared*

$$\frac{1}{2^2} = \frac{1}{4}, \quad \frac{1}{3^2} = \frac{1}{9}, \quad \frac{1}{6^2} = \frac{1}{36}$$

Total, over a common denominator of 36

$$\frac{9}{36} + \frac{4}{36} + \frac{1}{36} = \frac{14}{36}$$

Divide each by the total

$$u_1 = \frac{9}{14} = \mathbf{0.6429}, \quad u_2 = \frac{4}{14} = \mathbf{0.2857}, \quad u_3 = \frac{1}{14} = \mathbf{0.0714}$$

They sum to 1.0000. The hard label is **cluster 1**.

(c)

The second point, and the two numbers to compare are 0.6429 against 0.55. Neither is a strong membership — a decisive point runs above 0.9 — but the second point's winning cluster is more than twice its runner-up (0.6429 against 0.2857), while the first point's lead is 0.55 against 0.30. Both are worth flagging; neither should be treated as a settled segment member.

Note also what the $1/d^2$ weighting is doing. Distances of 2, 3 and 6 are not far apart, yet the memberships are 0.64, 0.29 and 0.07: squaring in the denominator makes the nearest centre dominate quickly, which is the same squaring that appears throughout k-Means. Figure 2 shows the same fact geometrically.

3 Part B — The k-Means objective

1. The concept

k-Means needs some way of saying that one grouping is better than another. It uses **inertia**, also called the within-cluster sum of squares: give every cluster a centre, measure how far each point sits from its own centre, square those distances, and add everything up. Small total, tight groups.

Squaring does three jobs. It discards the sign, so points on either side of a centre both count as being away from it. It makes the total smooth, so the best centre can be found by calculus instead of by trial and error. And it penalises one badly-placed point far more heavily than several slightly-off ones — which is the source of both the method's precision and its weakness to outliers.

The centre itself is the **centroid**: the mean of the points currently assigned to that cluster. It is a position in the feature space, and it need not be one of your data points.

2. The picture

Inertia is easiest to see as *spoke length*. Draw a line from every point to its own centroid; WCSS is the total of those lengths, squared. Figure 3 shows the same eight points grouped two ways, and what the squaring does.

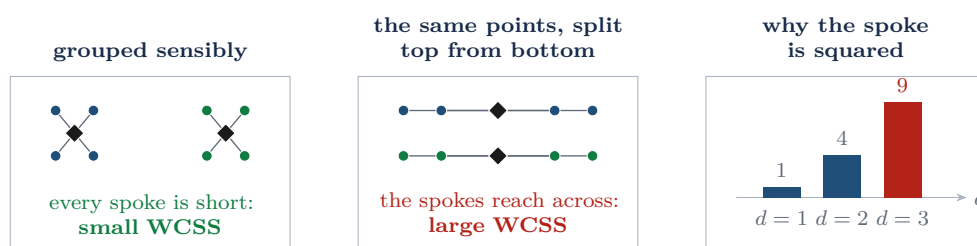


Figure 3: *Left and centre:* the same eight points, two groupings. The diamonds are the centroids and the thin lines are the distances WCSS adds up — inertia prefers the left because its spokes are shorter. *Right:* squaring is not neutral. It makes one badly-placed point cost more than several slightly-off ones, which is the source of both k-Means' precision and its weakness to outliers.

3. How it differs — squared distance against plain distance

Inertia is one choice of objective, not the only one. Change a single thing — drop the square — and you get the cost k-Medoids minimises instead. Everything that separates the two methods in Part D follows from this one line.

	Inertia (k-Means)	Medoid cost (k-Medoids)
What is added up	$[d(x_1, \mu)]^2 + \dots + [d(x_n, \mu)]^2$ every distance <i>squared</i>	$d(x_1, m) + \dots + d(x_n, m)$ every distance as it is
Where the centre may sit	anywhere at all — it is a position, not an observation	only on one of your data points
How the centre is found	by formula — it is the mean of the members	by search — try each member in turn, keep the cheapest
A point 10 away contributes	$10^2 = 100$	10
So one outlier	dominates the total and drags the centre with it	counts once, like every other point

The square is what makes the mean the answer, which is what makes the update step exact, which is what guarantees k-Means terminates. It is also the reason a single extreme value can wreck the result. You cannot keep one property and discard the other — they are the same square.

One caution about the medoid it is tempting to say “mean becomes median”, and that is not quite right. The medoid is defined by a *restriction* — the centre must be one of your observations — not by the median formula. The two only coincide by accident:

- On a **line**, the quantity $|x_1 - c| + \dots + |x_n - c|$ really is smallest at the median, and with an odd number of points the median is itself one of the observations. There the medoid and the median land on the same value.
- In **two or more dimensions** they part company. The free minimiser is the *geometric median*, a point that is usually not one of your observations and is not the median of each coordinate either. The medoid is the best *observation*, which sits near it but is not the same thing.

So: k-Means has a formula for its centre and k-Medoids does not. That is exactly why PAM has to test swaps rather than compute an update, and it is where the extra cost in Part D comes from.

4. The numerical

Worked example 3.1 — *computing inertia for a two-cluster grouping*

Take $P = \{(0, 0), (4, 0), (2, 3)\}$ and $Q = \{(9, 6), (11, 8)\}$.

Step 1 — find each centroid, the mean of its members

Add the coordinates and divide by how many points there are. Do x and y separately.

$$\mu_P = \left(\frac{0+4+2}{3}, \frac{0+0+3}{3} \right) = \left(\frac{6}{3}, \frac{3}{3} \right) = (2, 1)$$

$$\mu_Q = \left(\frac{9+11}{2}, \frac{6+8}{2} \right) = (10, 7)$$

Step 2 — squared distance from each point to its OWN centroid

For each point, subtract the centroid coordinate by coordinate, square both gaps, and add the two squares.

$$\begin{aligned} (0, 0) \text{ to } (2, 1) : & (0 - 2)^2 + (0 - 1)^2 = 4 + 1 = 5 \\ (4, 0) \text{ to } (2, 1) : & (4 - 2)^2 + (0 - 1)^2 = 4 + 1 = 5 \\ (2, 3) \text{ to } (2, 1) : & (2 - 2)^2 + (3 - 1)^2 = 0 + 4 = 4 \end{aligned}$$

$$\begin{aligned}(9, 6) \text{ to } (10, 7) : & \quad (9 - 10)^2 + (6 - 7)^2 = 1 + 1 = 2 \\ (11, 8) \text{ to } (10, 7) : & \quad (11 - 10)^2 + (8 - 7)^2 = 1 + 1 = 2\end{aligned}$$

Step 3 — add them all

$$\text{WCSS} = \underbrace{5 + 5 + 4}_{P = 14} + \underbrace{2 + 2}_{Q = 4} = \mathbf{18}$$

Why cluster Q gave two equal values

Worth noticing, because it happens constantly and it is not a coincidence. A cluster of *two* points is always symmetric: the mean of two numbers is their midpoint, so each point ends up exactly half the gap from the centre, and both contribute the same amount. You will only ever get *different* contributions inside a cluster of three or more — as *P* shows, where the lone high point (2, 3) contributes 4 while the two low ones contribute 5 each.

Use this as a check on your own arithmetic. If a three-point cluster gives you three identical numbers, look again.

Reading the answer

Eighteen means nothing on its own. It becomes useful the moment you compute it for a *different* grouping of the same points and compare the two — which is the only thing inertia is ever for.

Questions

B1 Inertia of a given clustering A clustering has produced two clusters:

$$C_1 = \{(1, 2), (3, 2), (2, 5)\}, \quad C_2 = \{(8, 7), (10, 9)\}.$$

Compute both centroids, then the total WCSS.

Solution.

Centroids

$$\mu_1 = \left(\frac{1+3+2}{3}, \frac{2+2+5}{3} \right) = (2, 3), \quad \mu_2 = \left(\frac{8+10}{2}, \frac{7+9}{2} \right) = (9, 8)$$

Squared distances in C_1

$$(1, 2): 1 + 1 = 2, \quad (3, 2): 1 + 1 = 2, \quad (2, 5): 0 + 4 = 4 \Rightarrow 8$$

Squared distances in C_2

$$(8, 7): 1 + 1 = 2, \quad (10, 9): 1 + 1 = 2 \Rightarrow 4$$

Total

$$\text{WCSS} = 8 + 4 = \mathbf{12}$$

Note the units: WCSS is a sum of *squared* distances, so it is not measured in the same units as the features.

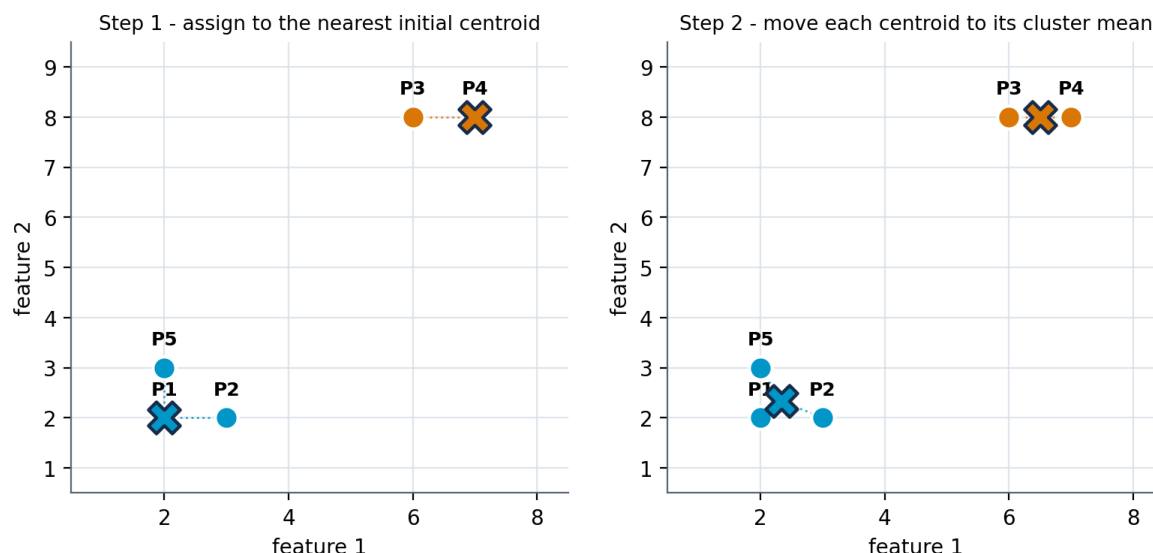


Figure 4: One iteration: assign every point to its nearest centroid, then move each centroid to the mean of what it collected.

B2 One full iteration of k-Means Five points are to be split into $k = 2$ clusters:

$$P_1(2, 2), \quad P_2(3, 2), \quad P_3(6, 8), \quad P_4(7, 8), \quad P_5(2, 3),$$

with initial centroids $C_1 = (2, 2)$ and $C_2 = (7, 8)$. Carry out one assignment step and one update step, then report the WCSS after the update. Figure 4 shows both steps.

Solution.

Assignment step — distance to each centroid

$$\begin{array}{lll} P_1 & d(C_1) = 0.0000 & d(C_2) = 7.8102 \rightarrow C_1 \\ P_2 & d(C_1) = 1.0000 & d(C_2) = 7.2111 \rightarrow C_1 \\ P_3 & d(C_1) = 7.2111 & d(C_2) = 1.0000 \rightarrow C_2 \\ P_4 & d(C_1) = 7.8102 & d(C_2) = 0.0000 \rightarrow C_2 \\ P_5 & d(C_1) = 1.0000 & d(C_2) = 7.0711 \rightarrow C_1 \end{array}$$

So $C_1 = \{P_1, P_2, P_5\}$ and $C_2 = \{P_3, P_4\}$.

Update step — each centroid becomes the mean of its members

$$\mu_1 = \left(\frac{7}{3}, \frac{7}{3}\right) = (2.3333, 2.3333), \quad \mu_2 = (6.5, 8)$$

WCSS after the update

$$C_1: 0.2222 + 0.5556 + 0.5556 = 1.3333, \quad C_2: 0.25 + 0.25 = 0.5$$

$$\text{WCSS} = \mathbf{1.8333}$$

B3 Why the mean is the right centre One cluster holds the m points $x_1, x_2, \dots, x_m$, and we must choose a single centre c to represent them. That cluster contributes

$$f(c) = \sum_{i=1}^m (x_i - c)^2$$

to the objective. Show that the value of c minimising f is the arithmetic mean, and confirm it really is a minimum.

Solution.

Differentiate with respect to c

$$f'(c) = \sum_{i=1}^m 2(x_i - c)(-1) = -2 \sum_{i=1}^m (x_i - c)$$

Set the derivative to zero

$$-2 \sum_{i=1}^m (x_i - c) = 0 \implies \sum_{i=1}^m (x_i - c) = 0$$

Split the summation — c is added m times

$$\sum_{i=1}^m x_i - m c = 0$$

Solve

$$c = \frac{1}{m} \sum_{i=1}^m x_i = \bar{x} \quad \text{the arithmetic mean}$$

Confirm it is a minimum

$$f''(c) = 2m > 0 \quad \text{for any non-empty cluster}$$

so the stationary point is a minimum, and being the only one, it is the global minimum.

What this buys you

The update step of k-Means is therefore *not* a heuristic — it is the exact minimiser of the objective, which is precisely why the objective can never increase and the algorithm is guaranteed to terminate. It is also why the method is called k-MEANS.

Replace the squared distance with the plain distance $|x_1 - c| + \dots + |x_m - c|$ and this neat argument stops working: the absolute value has no derivative at zero, so there is no formula to solve. On a line the answer turns out to be the **median**, but note what that is and is not. It is the best position on the line, found by an argument about counting, not by calculus — and in two or more dimensions the corresponding answer is the *geometric median*, which usually is not one of your data points at all.

k-Medoids goes one step further and *requires* its centre to be one of the observations. That restriction, not the median, is what defines a medoid, and it is why the centre has to be searched for rather than computed — see Part D.

B4 Cost of one iteration A dataset holds $n = 10,000$ points with $d = 5$ features, and k-Means is run with $k = 8$ for 12 iterations. How many point-to-centroid distance computations are performed per iteration, and in total? If each distance costs d coordinate operations, how many coordinate operations is that?

Solution.

$$\text{per iteration} = n \times k = 10,000 \times 8 = \mathbf{80,000}$$

$$\text{total} = 80,000 \times 12 = \mathbf{960,000}$$

$$\text{coordinate operations} = n k d \times 12 = 10,000 \times 8 \times 5 \times 12 = \mathbf{4,800,000}$$

This is the $O(n k d)$ per-iteration cost quoted for k-Means. It is linear in every one of the three, which is exactly why k-Means scales to data where most alternatives do not.

B5 Why the objective cannot be searched exhaustively Rather than iterating, why not simply try every possible assignment of n points to k clusters and keep the best? Count the assignments for $n = 20$, $k = 3$, and then for $n = 100$, $k = 3$.

Solution. Each of the n points may go to any of the k clusters independently, so there are k^n assignments.

$$3^{20} = \mathbf{3,486,784,401} \approx 3.5 \times 10^9$$

$$3^{100} \approx \mathbf{5.15 \times 10^{47}}$$

Twenty points already give three and a half billion possibilities. At a hundred points the count exceeds the number of atoms in the Earth. Exhaustive search is not slow here — it is impossible, which is why k-Means settles for an iterative method that reaches a *local* minimum quickly.

4 Part C — Initialisation and convergence

1. The concept

k-Means is guaranteed to stop, but not guaranteed to stop anywhere good. Every run settles at a **local minimum** — a grouping that cannot be improved by moving any single point or centre, but which may still be far worse than the best grouping available. Which local minimum you land in is decided entirely by where the centres started.

Random initialisation picks k data points at random. It is fast, and it regularly drops two centres inside the same natural group, leaving another group with none at all.

k-Means++ fixes that cheaply. The first centre is still random. Every later centre is also drawn at random, but the draw is weighted by $D(x)^2$ — the squared distance from each point to the nearest centre already chosen — so a point far from everything picked so far becomes overwhelmingly more likely to be selected.

2. The picture

The whole of this part is one picture. Two of the six points are drawn as starting centres; where they land decides the answer before a single iteration runs. Figure 5 shows the two outcomes.

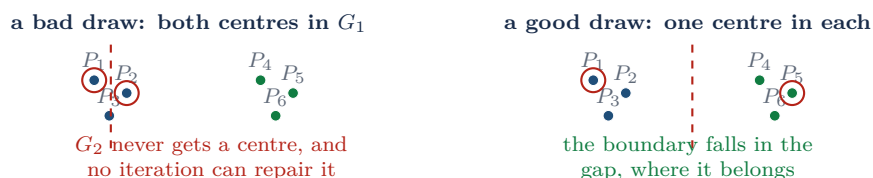


Figure 5: The two kinds of start. Circled points are the drawn centres and the dashed line is the boundary they induce. Both pictures are *converged* k-Means runs — which is the entire problem with convergence as a guarantee. Once both centres land in G_1 , no amount of iterating can move one across to G_2 .

3. How it differs — k-Means++ against random initialisation

Both draw the same number of starting centres from the same data. They differ only in the weighting of the draw, and that alone changes how often the left-hand picture above occurs.

	Random initialisation	k-Means++
How each centre is drawn	uniformly — every point equally likely	weighted by $[D(x)]^2$, the squared distance to the nearest centre chosen so far
Chance of a nearby point being picked next	the same as any other point	almost none — a small $D(x)$ gives a tiny weight
Two centres landing in one clump	happens routinely, even on clean data	very unlikely
Cost of the initialisation	essentially free	k passes over the data, once
Fixes the problem by	repeating the whole run (<code>n_init</code>) and keeping the best	making any <i>single</i> run unlikely to start badly

These are not rivals. `n_init` attacks a bad start by trying again; k-Means++ attacks it by making the first try good. Real implementations do both, which is why scikit-learn's defaults

are `init="k-means++"` and `n_init>1`.

4. The numerical

Worked example 4.1 — *k-Means++ probabilities for a second centre*

Three points lie on a line: $A(0,0)$, $B(1,0)$ and $C(5,0)$. The first centre chosen is A .

Step 1 — $D(x)$ is the distance to the nearest chosen centre, here A

$$D(B) = 1, \quad D(C) = 5$$

Step 2 — square them and total

$$D(B)^2 = 1, \quad D(C)^2 = 25, \quad \sum D^2 = 26$$

Step 3 — divide each by the total

$$\Pr(B) = \frac{1}{26} = 0.0385, \quad \Pr(C) = \frac{25}{26} = 0.9615$$

Reading the answer

C is chosen roughly 96 times in 100. A uniform random draw would have given B and C an equal 50% each — and choosing B would have placed both centres inside the same tight pair, leaving C stranded with no centre near it.

Questions

C1 k-Means++ initialisation Using the same five points, suppose k-Means++ has chosen $P_1(2,2)$ as its first centre. Compute $D(x)^2$ for each remaining point and hence the probability that each is chosen as the second centre. Which is most likely, and why is that the desired behaviour?

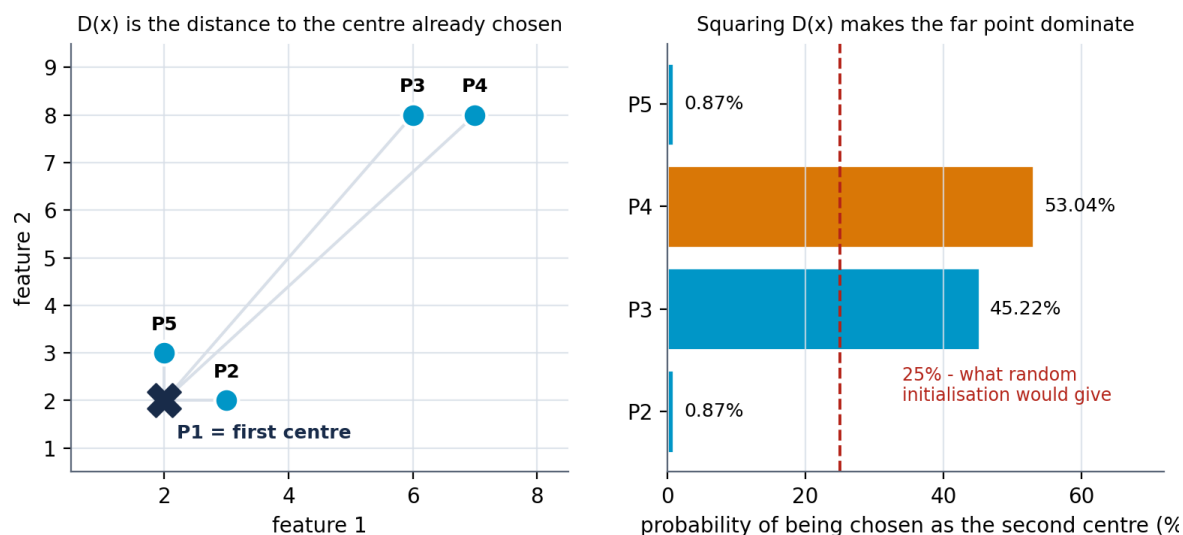


Figure 6: Squaring $D(x)$ pushes almost all the probability onto the far group, where a second centre is actually needed. Random initialisation would give every point an equal 25%.

Solution.

$D(x)^2$ is the squared distance to the only centre so far, $P_1(2, 2)$

$$\begin{array}{rcl} P_2(3, 2) & 1 + 0 & = 1 \\ P_3(6, 8) & 16 + 36 & = 52 \\ P_4(7, 8) & 25 + 36 & = 61 \\ P_5(2, 3) & 0 + 1 & = 1 \end{array}$$

Normalise

$$\begin{aligned} \sum D(x)^2 &= 1 + 52 + 61 + 1 = 115 \\ \Pr(P_2) &= \frac{1}{115} = 0.0087, \quad \Pr(P_3) = \frac{52}{115} = 0.4522, \\ \Pr(P_4) &= \frac{61}{115} = \mathbf{0.5304}, \quad \Pr(P_5) = \frac{1}{115} = 0.0087 \end{aligned}$$

Interpretation

P_4 is most likely at roughly 53%, and P_3 and P_4 together take 98% of the probability. Squaring the distance makes far-away points dominate the draw, so the second centre lands in the other natural group. Random initialisation gives all four an equal 25%, which is how two centres end up inside the same clump and the run converges somewhere poor.

C2 Choosing between two runs k-Means was run twice on the same six points, from different random initialisations:

$$(1, 1), \quad (1, 2), \quad (2, 1), \quad (8, 8), \quad (8, 9), \quad (9, 8).$$

Run A finished with the clusters

$$\{(1, 1), (1, 2), (2, 1)\} \quad \text{and} \quad \{(8, 8), (8, 9), (9, 8)\},$$

while run B finished with

$$\{(1, 1), (1, 2)\} \quad \text{and} \quad \{(2, 1), (8, 8), (8, 9), (9, 8)\}.$$

Compute the WCSS of each and say which run the algorithm keeps.

Solution.

Run A

Centroids (1.3333, 1.3333) and (8.3333, 8.3333); each cluster contributes 1.3333, so $\text{WCSS}_A = \mathbf{2.6667}$.

Run B

First cluster: centroid (1, 1.5), contributing $0.25 + 0.25 = 0.5$. Second cluster: centroid (6.75, 6.5), and the stray point (2, 1) alone contributes $(-4.75)^2 + (-5.5)^2 = 52.8125$; with the other three the cluster totals 71.75.

$$\text{WCSS}_B = 0.5 + 71.75 = \mathbf{72.25}$$

Conclusion

Run A wins by a factor of 27, and the algorithm keeps it. Both runs *converged* — convergence only ever guarantees a local minimum. This is precisely what `n_init` does: repeat the whole algorithm from several initialisations and retain the lowest inertia.

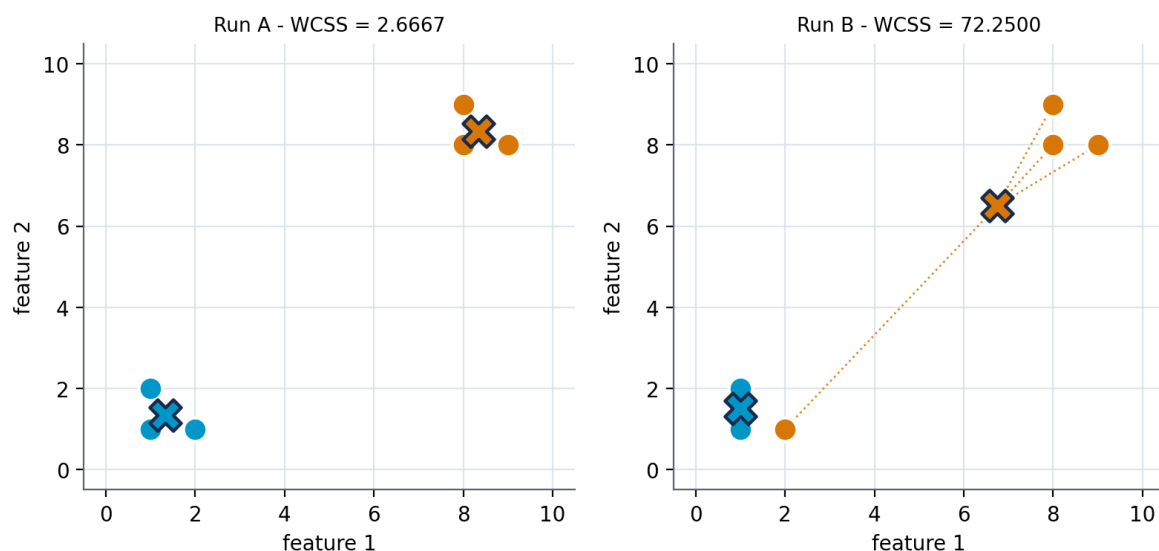


Figure 7: Both runs converged. Only one of them is any good — and inertia is the number that tells them apart.

C3 Reasoning about convergence Answer in one or two sentences each, without computing anything.

- k-Means is run twice on the same data with the same k and gives two different answers. Is that a bug?
- Why can k-Means never loop forever?
- Why can k-Means not recover two clusters shaped like concentric rings?

Solution. (a) No. k-Means converges to a *local* minimum of WCSS, and which one it reaches depends on the initial centroids. Different initialisations legitimately give different answers, which is why the run is repeated and the lowest inertia kept.

(b) Both steps can only reduce WCSS or leave it unchanged — the assignment step moves each point to a nearer centroid, and the update step moves each centroid to the point that minimises squared distance to its members. Since there are finitely many possible assignments and the objective never increases, the algorithm cannot cycle and must stop.

(c) Because it minimises squared distance to a centroid, every cluster it can produce is built around a central point and is therefore convex and roughly spherical. The centre of a ring is not in the ring, so no choice of centroids describes the rings. The limitation follows from the objective function itself, not from a poor implementation.

C4 How often random initialisation goes wrong Six points form two obvious groups of three,

$$G_1 = \{P_1, P_2, P_3\}, \quad G_2 = \{P_4, P_5, P_6\},$$

and $k = 2$. Plain random initialisation draws two of the six points uniformly at random, without replacement, and uses them as the starting centres. (a) How many different pairs can be drawn? (b) How many of those pairs put *both* centres inside the same group — the bad start of Figure 5? (c) Hence give the probability of a bad start, and then the probability that at least one of $n_{\text{init}} = 5$ independent restarts begins well.

Solution.

(a) *Every unordered pair of six points*

$$\binom{6}{2} = \frac{6 \times 5}{2} = \mathbf{15}$$

(b) *Pairs drawn from within one group*

$$\binom{3}{2} = 3 \text{ from } G_1, \quad \binom{3}{2} = 3 \text{ from } G_2, \quad \text{total } \mathbf{6}$$

(c) *The probabilities*

$$\Pr(\text{bad start}) = \frac{6}{15} = \mathbf{0.4}$$

$$\Pr(\text{all five restarts bad}) = 0.4^5 = \mathbf{0.01024}$$

$$\Pr(\text{at least one good start}) = 1 - 0.01024 = \mathbf{0.98976}$$

Reading the answer

Two points worth taking away. First, a bad start is not a rare accident: with data this clean it happens two times in five, and both centres landing in one group means the other group never gets one — no amount of iterating repairs that, because the run is already at a local minimum.

Second, this is exactly what `n_init` is for. Five independent restarts turn a 40% failure rate into a 1% one for five times the compute, and keeping the lowest-inertia run costs nothing extra. Compare this with the k-Means++ question that opened this part: k-Means++ attacks the same problem from the other end, by making any *single* start unlikely to be bad. Real implementations use both.

5 Part D — k-Medoids and MiniBatch k-Means

1. The concept

Two variants, each repairing a different weakness of k-Means.

k-Medoids changes what may count as a centre. Instead of the mean, the centre must be an actual member of the cluster — a **medoid** — and the cost being minimised uses plain distances rather than squared ones. Both changes cut the influence of extreme values: distances are no longer amplified by squaring, and the centre cannot drift into empty space because it has to sit on a real observation. **PAM** is the classical algorithm for it: repeatedly test swapping a medoid for a non-medoid, and keep the swap whenever the cost falls.

MiniBatch k-Means changes how much data each step looks at. Rather than using all n points to update the centres, it draws a small random batch each iteration. The centres then follow a noisier path and the final inertia is usually a little worse — but the cost per iteration stops depending on the size of the dataset, which is what makes millions of rows tractable.

2. The picture

Each variant changes one thing, and each change is visible. k-Medoids changes *where the centre may sit* (Figure 8); MiniBatch changes *how much data is looked at* (Figure 9).

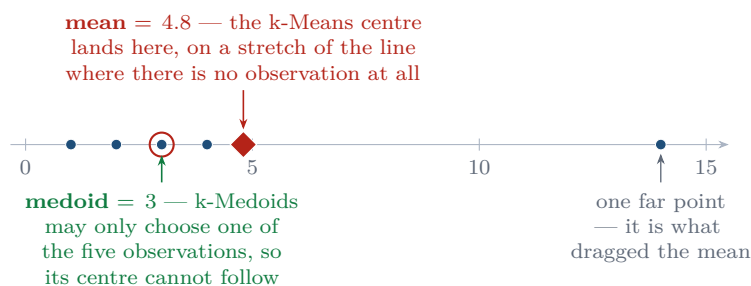


Figure 8: Five observations on a line. The mean is dragged into empty space by the far point; the medoid cannot follow it there, because it is required to be one of the observations. That single restriction is the whole of k-Medoids’ robustness.

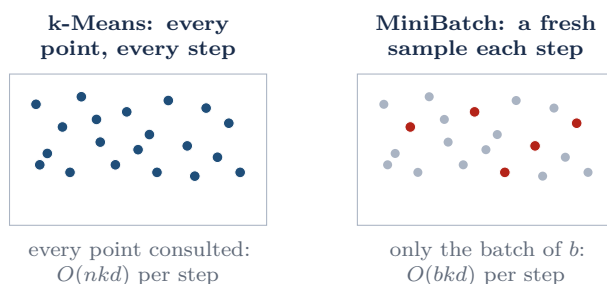


Figure 9: The same dataset, one iteration. Full k-Means consults every point at every step, so the cost per step grows with the size of the data. MiniBatch consults a random b of them, so it does not — and that is the entire reason millions of rows become tractable.

3. How it differs — the three side by side

All three are partition methods answering the same question — *group these points around k centres* — and they differ only in what a centre is allowed to be, and how much of the data is consulted to place it.

	k-Means	k-Medoids (PAM)	MiniBatch k-Means
The centre is	the mean — any position	one of your data points	the mean — any position
It adds up	every distance <i>squared</i>	every distance as it is	the squares, from a sample
An outlier	drags the centre without limit	cannot move the centre off the data	drags the centre, as k-Means
Work per iteration	$n \times k$ distances	every pair inside a cluster	$b \times k$ distances, and b is fixed
Comfortable up to	millions of rows	thousands of rows	effectively unlimited
You can point at the centre and say	“this is the average customer”	“this <i>is</i> customer C0132”	“this is the average customer”

Read the bottom two rows together and the trade is plain. k-Medoids buys robustness and interpretability with time; MiniBatch buys time with accuracy; k-Means buys neither and pays for neither, which is why it remains the default. Nothing here makes one of them correct — the dataset decides.

4. The numerical

Worked example 5.1 — finding the medoid of a small cluster

A cluster holds the points 1, 2 and 6.

Step 1 — test every member as a candidate centre

$$m = 1 : 0 + 1 + 5 = 6, \quad m = 2 : 1 + 0 + 4 = \mathbf{5}, \quad m = 6 : 5 + 4 + 0 = 9$$

Step 2 — keep the smallest

The medoid is 2, at a cost of 5.

Compare with what k-Means would have done

The mean is $(1 + 2 + 6)/3 = 3$ — a position with no data point on it at all. The medoid is a genuine member of the cluster, which is precisely what makes it something you can point at and show to somebody.

Questions

D1 Why a medoid survives an outlier A one-dimensional cluster contains 2, 3, 4, 5, 100. Compute the k-Means centre (the mean) and the k-Medoids centre (the member minimising the sum of absolute distances). Which would you report to a business, and why?

Solution.

The mean, which is what k-Means would use

$$\mu = \frac{2 + 3 + 4 + 5 + 100}{5} = \frac{114}{5} = \mathbf{22.8}$$

The medoid — test every member as a candidate

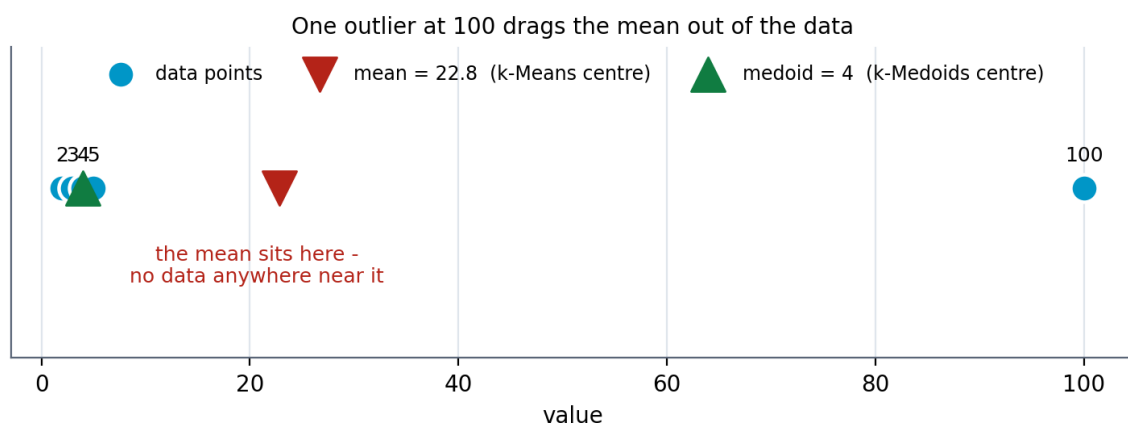


Figure 10: A single outlier is enough to move the mean into empty space. The medoid cannot move there, because it is required to be one of the observations.

$$\begin{aligned}
 m = 2 : & \quad 0 + 1 + 2 + 3 + 98 = 104 \\
 m = 3 : & \quad 1 + 0 + 1 + 2 + 97 = 101 \\
 m = 4 : & \quad 2 + 1 + 0 + 1 + 96 = \mathbf{100} \quad (\text{lowest}) \\
 m = 5 : & \quad 3 + 2 + 1 + 0 + 95 = 101 \\
 m = 100 : & \quad 98 + 97 + 96 + 95 + 0 = 386
 \end{aligned}$$

So the medoid is **4**, with cost 100.

Interpretation

The single outlier drags the mean to 22.8 — a value describing none of the five points, with no data anywhere near it. The medoid stays at 4 because it must be an actual observation and the cost does not square distances. Report the medoid: it is a real customer you could point at.

D2 Evaluating a medoid swap A cluster holds 1, 2, 3, 8, 9 and its current medoid is 2. PAM proposes swapping the medoid to 3. Compute the cost before and after, and state whether the swap is accepted.

Solution.

Current medoid $m = 2$

$$|1 - 2| + |2 - 2| + |3 - 2| + |8 - 2| + |9 - 2| = 1 + 0 + 1 + 6 + 7 = \mathbf{15}$$

Proposed medoid $m = 3$

$$|1 - 3| + |2 - 3| + |3 - 3| + |8 - 3| + |9 - 3| = 2 + 1 + 0 + 5 + 6 = \mathbf{14}$$

Decision

The cost falls by 1, so the swap is **accepted**. PAM repeats exactly this test for every (medoid, non-medoid) pair and applies the swap that helps most, stopping when no swap improves the cost. That is why classic PAM is so much more expensive than k-Means — it evaluates $k(n - k)$ candidate swaps per step rather than computing a mean.

- D3 MiniBatch k-Means** (a) A dataset holds 1,000,000 points. MiniBatch k-Means runs for 50 iterations with a batch size of 2,000. How many point-evaluations does it perform, how many would full-batch k-Means perform over the same 50 iterations, and what is the ratio?
- (b) On the same data full-batch k-Means finishes at $WCSS = 1,180$ while MiniBatch finishes at 1,239. Express the quality lost as a percentage, and say whether you would accept the trade.

Solution.

(a) *Point-evaluations*

$$\text{MiniBatch} = 50 \times 2,000 = \mathbf{100,000}$$

$$\text{Full batch} = 50 \times 1,000,000 = \mathbf{50,000,000}$$

$$\text{Ratio} = \mathbf{500} \times \text{fewer}$$

That saving is what makes clustering at this scale practical. The cost is that each update is computed from a sample, so the centroids follow a noisier path and the final inertia is usually a little worse than full k-Means would reach.

(b) *What the noisier path costs*

$$\frac{1,239 - 1,180}{1,180} = \frac{59}{1,180} = 0.05 = \mathbf{5\%} \text{ worse}$$

Accept it. Five per cent of inertia is a small and *measurable* loss, while a five-hundred-fold reduction in work is often the difference between a job that finishes and one that does not. Note the shape of the trade: MiniBatch can never win on quality, because it optimises the same objective using less information. The only question is ever how much quality you are buying the time with — and you can answer it exactly, by running both on a sample first.

6 Part E — Standardisation and scaling

1. The concept

Clustering is built entirely on distance, and distance adds a contribution from every feature. So a feature recorded on a numerically larger scale contributes larger gaps and quietly decides the outcome — whether or not it is the more important feature. The units you happened to record in become an unstated modelling choice.

That is the whole of this part. An income measured in rupees rather than thousands of rupees is a thousand times larger as a number, so it is a million times larger once squared — and every other feature in the dataset stops mattering. Nothing warns you, because nothing is wrong: the arithmetic is exactly what you asked for.

2. The picture

Two ways of seeing it. Figure 11 shows, on the left, how the squared distance between two customers is divided up between the features; and on the right, that this is not cosmetic — it changes which cluster a customer is put in.

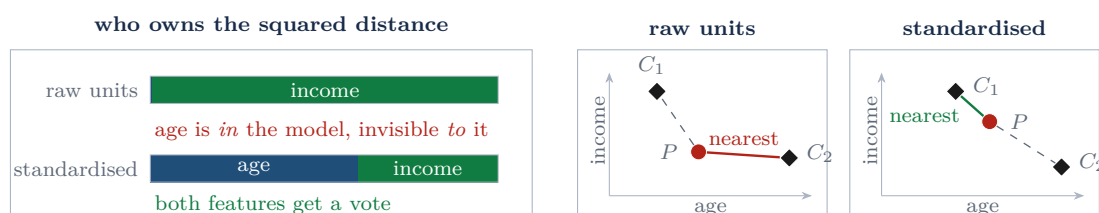


Figure 11: *Left:* the split is qualitative here — the exact percentages for one pair of customers are computed in step 4. *Right:* the same customer P and the same two centroids, measured twice. On raw units only income is visible, so P joins C_2 ; standardised, age is allowed to speak and P joins C_1 . Nothing about the customer changed — only the units.

3. How it differs — standardisation against min–max scaling

Both put features on a comparable footing. They differ in what they hold fixed, and therefore in how they treat an extreme value.

	Standardisation (z)	Min–max scaling
Formula	$z = \frac{x - \mu}{\sigma}$	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$
In words	how many standard deviations the value sits from its own average	how far along the range the value sits, from bottom to top
Divides by	the standard deviation — a <i>typical</i> gap	the full range — the gap between the two <i>extremes</i>
After scaling	average 0, spread 1; no upper limit	every value lies between 0 and 1
One extreme value	inflates σ a little; the outlier stays visibly large	becomes $x_{\max}$, squashing every ordinary value down near 0
Prefer it when	the features are roughly bell-shaped, or outliers matter	you need a bounded range and the data is clean

The fourth row is the one that decides it in practice. Min–max hands control of the entire scale

to a single observation — the largest one — so one mistyped income of 10^9 compresses every real customer into the bottom thousandth of the range, and the clustering that follows sees no differences between them at all. Standardisation divides by a typical gap instead, so an outlier merely comes out as a large z : still there, still visible, no longer in charge.

4. The numerical

Worked example 6.1 — what scaling does to a single distance

Two records hold (age, income): $r_1 = (20, 30000)$ and $r_2 = (30, 31000)$.

Step 1 — squared gaps on the raw numbers

$$\begin{aligned}\text{age} : (20 - 30)^2 &= 100, & \text{income} : (30000 - 31000)^2 &= 1,000,000 \\ d &= \sqrt{100 + 1,000,000} = 1000.05\end{aligned}$$

Age contributes $100/1,000,100$, which is 0.01% of the squared distance.

Step 2 — standardise, taking $\sigma_{age} = 5$ and $\sigma_{inc} = 1000$

$$\begin{aligned}\text{age gap} : \frac{10}{5} &= 2, & \text{income gap} : \frac{1000}{1000} &= 1 \\ d &= \sqrt{2^2 + 1^2} = \sqrt{5} = 2.2361\end{aligned}$$

Age now contributes $4/5$, which is 80%.

Reading the answer

Nothing about the two records changed — only the units did. Age went from contributing one part in ten thousand to being the dominant feature. That is why scaling is not housekeeping: it is a decision about what your clustering is permitted to see.

Questions

E1 Standardisation and min-max scaling (a) A feature has $\mu = 50$ and $\sigma = 12$. Compute the z-scores of 26, 50 and 74, and say what $z = 0$ means. (b) A second feature ranges from 15 to 95. Min-max scale the value 35. (c) Name one situation in which min-max is the worse choice.

Solution. (a)

$$z_{26} = \frac{-24}{12} = -2.0, \quad z_{50} = 0.0, \quad z_{74} = \frac{24}{12} = 2.0$$

A z-score of 0 means the observation sits exactly at the mean. The units are standard deviations, so 26 lies two of them below average.

(b) $x' = \frac{35 - 15}{95 - 15} = \frac{20}{80} = 0.25$

(c) When the feature contains outliers. A single extreme value sets $x_{\max}$, and every ordinary observation is then squashed into a narrow band near zero. Standardisation is less affected because it divides by the standard deviation rather than the full range.

E2 Scaling changes the answer A customer P has age 30 and income 40,000. Two centroids are

$$C_1 = (\text{age } 25, \text{ income } 41,000), \quad C_2 = (\text{age } 45, \text{ income } 39,800).$$

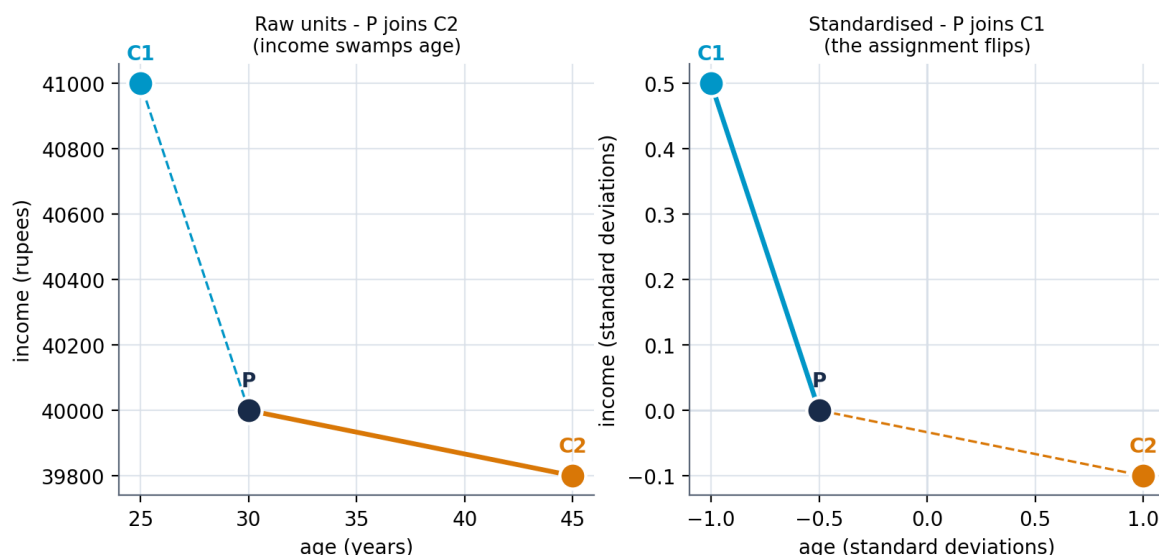


Figure 12: The same three points, twice. On raw units income is the only feature that exists; standardised, age is allowed to speak and the assignment reverses.

(a) Assign P using raw Euclidean distance. (b) Standardise with $\mu_{\text{age}} = 35$, $\sigma_{\text{age}} = 10$, $\mu_{\text{inc}} = 40,000$, $\sigma_{\text{inc}} = 2,000$, and assign again. (c) Explain the disagreement.

Solution.

(a) *Raw distances*

$$d(P, C_1) = \sqrt{25 + 1,000,000} = 1000.0125$$

$$d(P, C_2) = \sqrt{225 + 40,000} = 200.5617$$

P is assigned to C_2 .

(b) *Standardise, then measure again*

$$z(P) = (-0.5, 0), \quad z(C_1) = (-1.0, 0.5), \quad z(C_2) = (1.0, -0.1)$$

$$d(P, C_1) = \sqrt{0.5} = 0.7071, \quad d(P, C_2) = \sqrt{2.26} = 1.5033$$

P is now assigned to C_1 . **The assignment has flipped.**

(c) *Explanation*

Euclidean distance squares the raw gaps. The income difference of 1,000 contributes 1,000,000 while the age difference of 5 contributes 25 — so age is present in the model and has effectively no influence at all. Standardising puts both features on a common scale of standard deviations, and age is then able to matter. The unit a feature happened to be recorded in must never decide the clustering.

7 Part F — Cluster validation

1. The concept

Having produced clusters, you have to judge them — and with no labels there is no accuracy to compute. What is available are **internal** measures, which examine the geometry of the result and nothing else.

Inertia measures compactness alone: how tightly points sit around their centres. Its weakness is structural rather than practical. Adding a cluster can never make it worse, so it falls as k rises and reaches exactly zero when every point is its own cluster. It therefore cannot be minimised to choose k . It can only be inspected for a bend — the **elbow method** — and that bend is frequently absent, or read differently by different people.

The **Silhouette coefficient** is scored one point at a time, and it asks the question the point itself would ask: *am I in the right cluster?* It compares $a(i)$, the mean distance to the point's own clustermates, against $b(i)$, the mean distance to the members of the nearest other cluster. Read the three cases straight off the formula:

- $b \gg a$ — much closer to its own group than to any other, so $s(i)$ approaches +1: comfortably placed.
- $b \approx a$ — equally close to two clusters, so $s(i) \approx 0$: sitting on a boundary, and the label is a coin toss.
- $b < a$ — *closer to another cluster than to its own*, so $s(i) < 0$: misassigned. No other measure in this unit can say this about an individual point.

Dividing by $\max\{a, b\}$ is only normalisation, forcing the answer into $[-1, +1]$. Because s uses separation as well as compactness it does not fall forever as k rises — cutting a real group in two damages b — so it has a genuine maximum and may legitimately be optimised.

The **Davies–Bouldin Index** measures compactness *and* separation together, but per *pair of clusters* rather than per point. For each pair it compares their combined spread against the distance between their centres. Each cluster is then scored by its *worst* partner, and those scores are averaged. Lower is better, and unlike inertia it has a genuine minimum over k , so it can actually be optimised.

2. The picture

Start with the silhouette scale itself — Figure 13 is the whole of it at a glance, and the negative half is the part no other measure in this unit has.

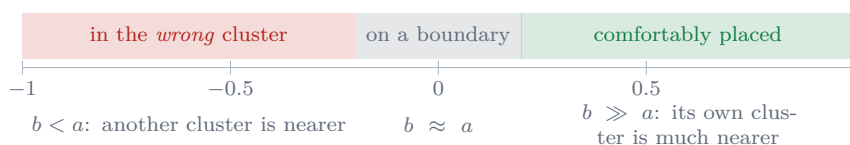


Figure 13: The silhouette scale. The negative half is what makes this measure different from the other two: it can tell you that an individual point has been put in the wrong cluster.

Then the difference between the three measures, which is easiest to see in the *shape* each one traces as k grows. Figure 14 is the picture to carry into the exam: inertia has no optimum to find, and the other two do.

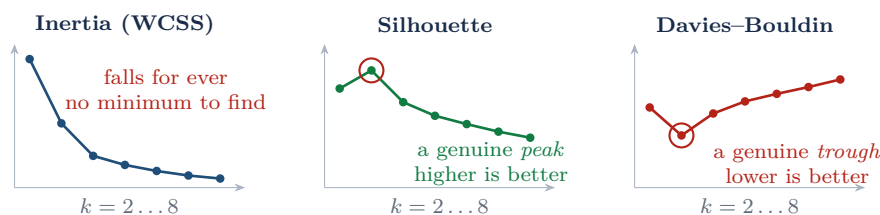


Figure 14: The same clustering, judged three ways. Inertia can only be *inspected* for a bend; silhouette and Davies–Bouldin can be *optimised*, and here both pick out the same k . They will not always agree, and when they disagree neither is wrong — they are measuring different things.

3. How it differs — the three measures side by side

All three judge geometry and nothing else — there are no labels to check against. They differ in *what* geometry they look at, and in whether the number they produce can be optimised.

	Inertia (WCSS)	Silhouette	Davies–Bouldin
It measures	compactness only	compactness <i>and</i> separation	compactness <i>and</i> separation
Scored per	the whole clustering	each point	each pair of clusters
It runs from	0 upwards, and lower is better	−1 to +1, and higher is better	0 upwards, and lower is better
As k rises it	falls for ever, reaching 0 when $k = n$	rises to a peak, then falls	dips to a trough, then rises
So to choose k you	<i>inspect</i> it for a bend	optimise it	optimise it
Can it name a wrongly-placed point?	no	yes — its score comes out negative	no

Two rows carry the whole part. The fifth explains why the elbow method exists at all: inertia has no minimum, so it cannot be optimised and must be read by eye. The sixth is the silhouette’s monopoly — it is the only measure here scored per point, so it is the only one that can hand you the identity of the customer in the wrong segment rather than a verdict on the segmentation as a whole.

4. The numerical

Worked example 7.1 — the silhouette of a single point

Cluster $A = \{(0, 0), (0, 1), (1, 0)\}$ and cluster $B = \{(4, 0), (4, 1)\}$. Take the point $i = (0, 0)$, which is labelled A .

Step 1 — $a(i)$: mean distance to its own clustermates, excluding itself

$$d((0, 0), (0, 1)) = 1, \quad d((0, 0), (1, 0)) = 1$$

$$a(i) = \frac{1+1}{2} = 1$$

Step 2 — $b(i)$: mean distance to the nearest other cluster

$$d((0, 0), (4, 0)) = 4, \quad d((0, 0), (4, 1)) = \sqrt{16 + 1} = 4.1231$$

$$b(i) = \frac{4+4.1231}{2} = 4.0616$$

Step 3 — combine

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} = \frac{4.0616 - 1}{4.0616} = \frac{3.0616}{4.0616} = \mathbf{0.7538}$$

Reading the answer

Comfortably above zero, so this point is where it belongs. Note that with only two clusters there is no choice about which one supplies $b(i)$; with three or more you must compute the mean distance to *each* other cluster and keep the smallest.

Worked example 7.2 — Davies–Bouldin for two clusters on a line

Take $C_1 = \{0, 2\}$ and $C_2 = \{8, 10\}$.

Step 1 — centroids

$$\mu_1 = \frac{0+2}{2} = 1, \quad \mu_2 = \frac{8+10}{2} = 9$$

Step 2 — scatter: mean distance from members to their own centroid

$$S_1 = \frac{|0-1|+|2-1|}{2} = 1, \quad S_2 = \frac{|8-9|+|10-9|}{2} = 1$$

Step 3 — separation between the centroids

$$M_{12} = |9 - 1| = 8$$

Step 4 — the ratio, then average over the clusters

$$R_{12} = \frac{S_1 + S_2}{M_{12}} = \frac{2}{8} = 0.25, \quad \text{DB} = \mathbf{0.25}$$

Reading the answer

A spread of 1 against a gap of 8 gives a small index, which is what a good clustering looks like. Push the two clusters closer together, or loosen either one, and the index rises immediately.

Questions

F1 Davies–Bouldin Index with two clusters For $C_1 = \{(1, 1), (2, 2), (1, 2)\}$ and $C_2 = \{(8, 8), (9, 9), (8, 9)\}$, compute the centroids, the scatters S_1 and S_2 , the separation M_{12} , and hence the Davies–Bouldin Index.

Solution.

Centroids

$$\mu_1 = (1.3333, 1.6667), \quad \mu_2 = (8.3333, 8.6667)$$

Scatter — mean distance of members to their own centroid

$$(1, 1): 0.7454, \quad (2, 2): 0.7454, \quad (1, 2): 0.4714$$

$$S_1 = \frac{0.7454 + 0.7454 + 0.4714}{3} = \mathbf{0.6540}, \quad S_2 = \mathbf{0.6540} \text{ by symmetry}$$

Separation

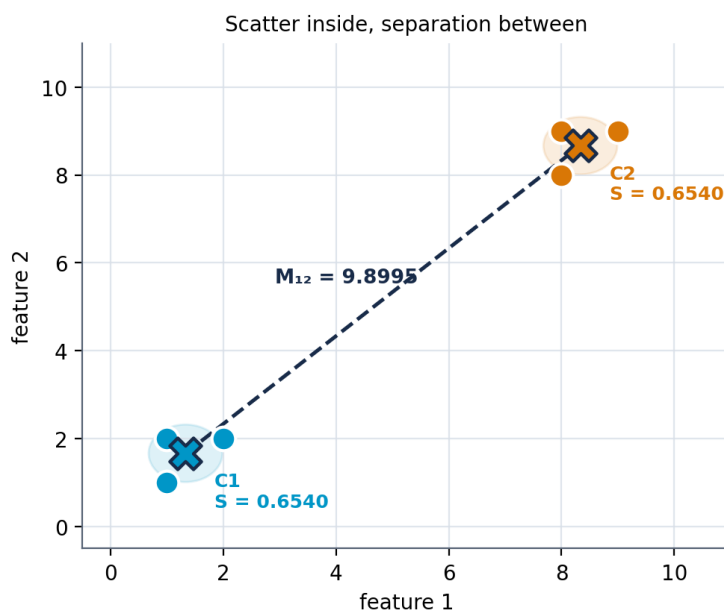


Figure 15: S measures spread inside a cluster; M measures the gap between clusters. Davies–Bouldin is the ratio.

$$M_{12} = \sqrt{49 + 49} = \sqrt{98} = 9.8995$$

Index

$$R_{12} = \frac{1.3080}{9.8995} = 0.1321, \quad \text{DB} = \frac{1}{2}(0.1321 + 0.1321) = \mathbf{0.1321}$$

A value this close to 0 says the clusters are tight relative to how far apart they sit. With $k = 2$ the max in the formula does nothing, because each cluster has only one partner — the next question fixes that.

F2 Davies–Bouldin with three clusters Now

$$C_1 = \{(0, 0), (2, 0)\}, \quad C_2 = \{(10, 0), (12, 0)\}, \quad C_3 = \{(6, 8), (6, 10)\}.$$

Each scatter works out to $S_j = 1$. Compute M_{12} , M_{13} , M_{23} , then all three R_{ij} , then apply the max correctly and report DB.

Solution.

Centroids and separations

$$\begin{aligned} \mu_1 &= (1, 0), & \mu_2 &= (11, 0), & \mu_3 &= (6, 9) \\ M_{12} &= 10.0000, & M_{13} &= \sqrt{25 + 81} = 10.2956 = M_{23} \end{aligned}$$

The ratios

$$R_{12} = \frac{1 + 1}{10.0000} = 0.2000, \quad R_{13} = R_{23} = \frac{2}{10.2956} = 0.1943$$

Take the worst partner for each cluster

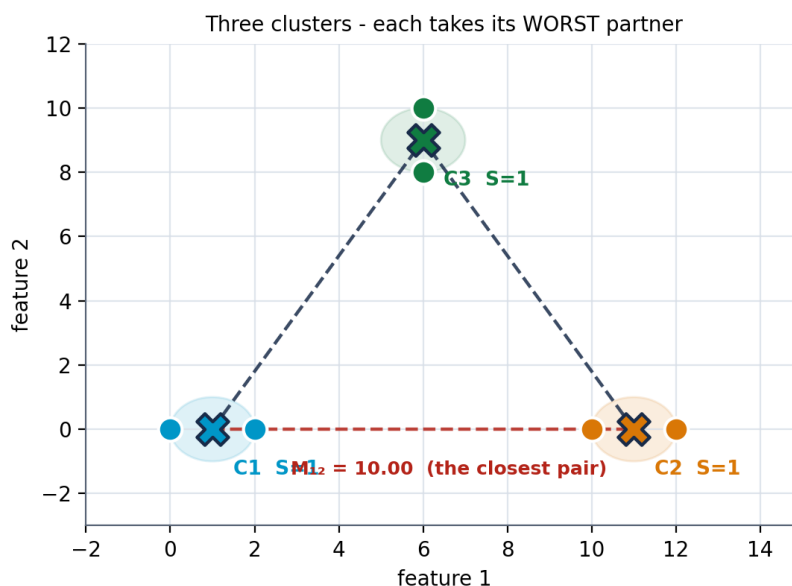


Figure 16: With three clusters each one is judged by its *worst* partner — the nearest, relative to their combined spread.

$$j = 1 : \max(0.2000, 0.1943) = 0.2000$$

$$j = 2 : \max(0.2000, 0.1943) = 0.2000$$

$$j = 3 : \max(0.1943, 0.1943) = 0.1943$$

Average over the clusters

$$DB = \frac{0.2000 + 0.2000 + 0.1943}{3} = \mathbf{0.1981}$$

Why the max matters

Clusters 1 and 2 are each other's worst problem — they are the closest pair, so both take 0.2000. Cluster 3 is comfortably away from both and takes the smaller 0.1943. The max means a clustering is judged by its most confusable pair, not by its average pair, which is why one badly-placed cluster is enough to spoil the index.

F3 The elbow method, and why inertia cannot be minimised Running k-Means for several k gives:

k	1	2	3	4	5	6
WCSS	1200	480	210	190	178	170

(a) Tabulate the drop at each step and identify the elbow. (b) Explain why choosing the k with the smallest WCSS is not an option.

Solution.

(a) *Successive drops*

k	2	3	4	5	6
drop	720	270	20	12	8

The drops collapse from 270 to 20 after $k = 3$, so the elbow is at $\mathbf{k = 3}$.

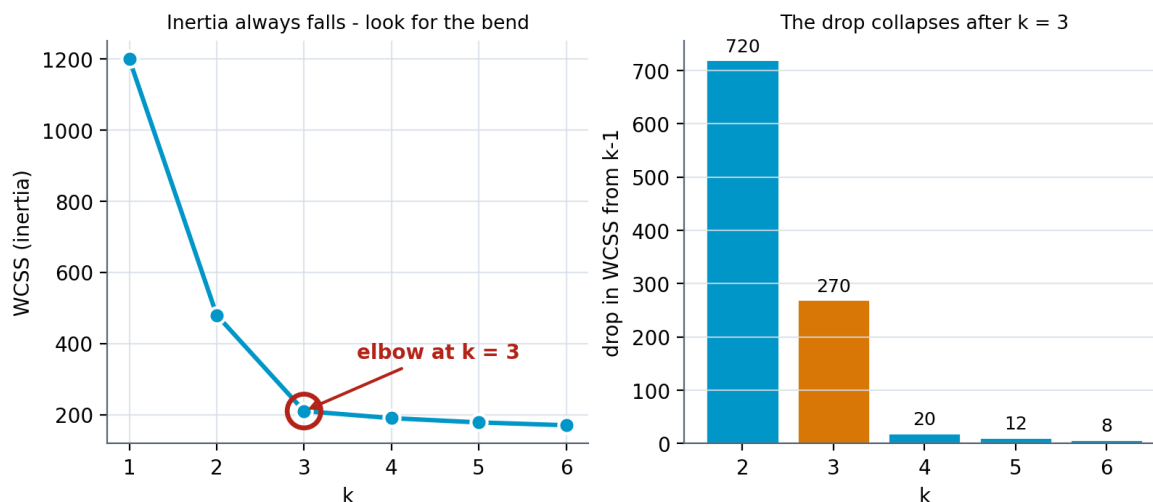


Figure 17: Inertia never rises, so it has no minimum to find. The bend is read from where the improvement stops being worth it.

(b) *Why not simply minimise*

WCSS falls monotonically as k rises — adding a centroid can never increase it. At $k = n$ every point is its own centroid, every distance is zero and $\text{WCSS} = 0$. Minimising WCSS therefore always returns $k = n$, which is not a clustering at all. Inertia can only be *inspected* for a bend, never optimised, and that is exactly why the Davies–Bouldin Index — which does have a genuine minimum — is worth computing beside it.

F4 Reading a disagreement On the same dataset the elbow reads $k = 3$, while Davies–Bouldin is lowest at $k = 6$. The marketing team can run three campaigns. What do you deliver, and what do you report alongside it?

Solution. Deliver $k = 3$, and say plainly why.

Both numbers are honest. The elbow is subjective and Davies–Bouldin has a genuine optimum, so on geometry alone $k = 6$ has the better case. But neither index can tell you the clusters are *meaningful* — both judge geometry only, and a segmentation that cannot be acted on has no value whatever its index.

Report three things: that $k = 3$ was chosen for operational reasons rather than geometric ones, the Davies–Bouldin value at both $k = 3$ and $k = 6$ so the cost of the choice is visible, and the recommendation that if marketing capacity later rises to six campaigns the $k = 6$ solution is the one to revisit.

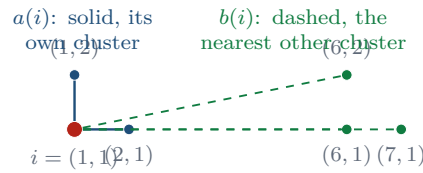
F5 The silhouette of a point Two clusters have been produced:

$$C_1 = \{(1, 1), (2, 1), (1, 2)\}, \quad C_2 = \{(6, 1), (7, 1), (6, 2)\}.$$

(a) Compute $a(i)$, $b(i)$ and $s(i)$ for the point $i = (1, 1)$, and say in one sentence what the value means. Figure 18 draws every distance the calculation needs. (b) Without computing it in full, will the point $(1, 2)$ have a higher or a lower silhouette than $(1, 1)$? Justify your answer from the formula, then check it.

Solution.

(a) $a(i)$ — mean distance to its own clustermates



$s(i)$ compares the average solid length against the average dashed length

Figure 18: What the silhouette actually measures for one point. Every distance in the calculation is drawn.

$$d((1, 1), (2, 1)) = 1, \quad d((1, 1), (1, 2)) = 1$$

$$a(i) = \frac{1+1}{2} = 1$$

$b(i)$ — mean distance to the members of C_2

$$d((1, 1), (6, 1)) = 5, \quad d((1, 1), (7, 1)) = 6,$$

$$d((1, 1), (6, 2)) = \sqrt{25 + 1} = 5.0990$$

$$b(i) = \frac{5 + 6 + 5.0990}{3} = \frac{16.0990}{3} = \mathbf{5.3663}$$

$s(i)$

$$s(i) = \frac{5.3663 - 1}{\max\{1, 5.3663\}} = \frac{4.3663}{5.3663} = \mathbf{0.8137}$$

Comfortably close to +1: this point is more than five times nearer to its own cluster than to the other one, so its label is not in doubt.

(b) *The prediction*

Lower. The point (1, 2) sits at the top corner, so it is further from its own cluster mate (2, 1) — a rises — while its distances to C_2 barely change, because C_2 is far away in the x direction and the shift is in y . A larger a with an almost unchanged b must give a smaller s .

Checking it

$$a = \frac{d((1, 2), (1, 1)) + d((1, 2), (2, 1))}{2} = \frac{1 + 1.4142}{2} = 1.2071$$

$$b = \frac{5.0990 + 6.0828 + 5}{3} = 5.3939$$

$$s = \frac{5.3939 - 1.2071}{5.3939} = \mathbf{0.7762}$$

Lower than 0.8137, as predicted — and note that a rose by 21% while b rose by only 0.5%. The corner point is the weaker member of its cluster, which is exactly the kind of thing a per-point measure exists to reveal.

F6 A negative silhouette A clustering has produced

$$C_1 = \{(0,0), (1,0), (8,0)\}, \quad C_2 = \{(9,0), (10,0), (11,0)\}.$$

(a) Compute $s(i)$ for the point $i = (8,0)$. (b) Compute what $s(i)$ would become if that point were moved to C_2 . (c) What does the sign of the answer to (a) tell you that the Davies–Bouldin Index cannot?

Solution.

(a) *The point is currently in C_1*

$$\begin{aligned} a(i) &= \frac{|8-0| + |8-1|}{2} = \frac{8+7}{2} = 7.5 \\ b(i) &= \frac{|8-9| + |8-10| + |8-11|}{3} = \frac{1+2+3}{3} = 2 \\ s(i) &= \frac{2-7.5}{\max\{7.5, 2\}} = \frac{-5.5}{7.5} = -\mathbf{0.7333} \end{aligned}$$

(b) *The same point, moved to C_2*

The two roles swap exactly:

$$\begin{aligned} a(i) &= \frac{1+2+3}{3} = 2, & b(i) &= \frac{8+7}{2} = 7.5 \\ s(i) &= \frac{7.5-2}{7.5} = +\mathbf{0.7333} \end{aligned}$$

(c) *What the sign buys you*

A negative silhouette is a statement about an *individual point*: this one is on average 7.5 from the cluster it was given and 2 from a cluster it was not, so it is in the wrong one. Part (b) confirms it — the same point scores +0.7333 in C_2 , and the clustering improves by moving it.

The Davies–Bouldin Index cannot say this. It is computed from centroids and average scatters, so it produces *one number for the whole clustering* and has no per-point component at all. It would report that these two clusters overlap badly, which is true but not actionable; only the silhouette names the point responsible. That is the practical reason to plot per-point silhouettes rather than quoting the average.

F7 Why the average silhouette hides things A clustering of 200 customers into four segments reports:

cluster	0	1	2	3
size	80	70	30	20
mean s	0.62	0.58	0.71	-0.05

(a) Compute the overall mean silhouette of the clustering. (b) Which cluster is the problem, and what does its value mean in plain words? (c) A colleague averages the four cluster means instead, unweighted. What do they get, and which of the two numbers would you put in the report?

Solution.

(a) *The silhouette of a clustering is the mean over points, so weight by size*

$$\frac{80(0.62) + 70(0.58) + 30(0.71) + 20(-0.05)}{200} = \frac{49.6 + 40.6 + 21.3 - 1.0}{200} \\ = \frac{110.5}{200} = \mathbf{0.5525}$$

(b) *Cluster 3*

Its mean is *negative*. On average its twenty members are closer to some other cluster than to their own, which means this is not a segment at all — it is twenty customers the algorithm had to put somewhere. The headline 0.5525 reads as “reasonable structure” on the conventional scale and conceals this completely, because the other 180 customers outvote them.

(c) *The unweighted average*

$$\frac{0.62 + 0.58 + 0.71 - 0.05}{4} = \frac{1.86}{4} = \mathbf{0.465}$$

Lower, because the failing cluster is small and unweighted averaging gives it the same say as the cluster of eighty.

Report the weighted 0.5525, since that is what `silhouette_score` computes and what a reader will assume you mean — but do not report it alone. The number to put beside it is the per-cluster table itself, because the decision it drives is “drop or merge cluster 3 and re-run”, and neither average says that on its own.

Syllabus map. Part A — clustering fundamentals and the partition property (A1), the assumptions k-Means makes and the limitations that follow (A2), hard versus soft clustering (A3). Part B — the k-Means objective function (B1, B2), why the mean is its exact minimiser (B3), cost and why exhaustive search is impossible (B4, B5). Part C — initialisation strategies: k-Means++ (C1) and random with `n_init` (C4), convergence and local minima (C2, C3). Part D — k-Medoids and PAM (D1, D2), MiniBatch k-Means and its quality cost (D3). Part E — data standardisation and scaling impact (E1, E2). Part F — cluster validation with inertia and the elbow method with its pitfalls (F3, F4), the Davies–Bouldin Index (F1, F2), and the silhouette coefficient (F5, F6, F7).